Air University Islamabad • Department of Computer Science

FSD Lab 05 Tasks

jQuery, AJAX, Node.js & Express.js Lab Assignment

Student Name	**Hamza Tahir**
Registration ID	**225193**
Course	**Full Stack Development (CS-301L)**
Lab Session	**Week 5 — Lab 05**
Submission Date	**October 6, 2026**
Technology Stack	**HTML, CSS, jQuery 3.7.1, Node.js, Express**

Task 1 Interactive Product Configurator (jQuery DOM Events & Selectors)

Task 2 Public Product API Integration (jQuery AJAX GET Request)

Task 3 Student Management System (Express REST API + jQuery POST)

TASK 1 / QUESTION 1

Interactive Product Configurator — jQuery DOM Manipulation

Demonstrates jQuery selectors, event handling, real-time DOM updates, and dynamic class manipulation

📋 Task Overview

Build an interactive product configurator using jQuery. The configurator allows users to customize a Smart Thermal Tumbler by selecting a color finish, choosing the size/capacity, adding a laser engraving, and optionally unlocking premium features via a simulated purchase. All interactions are handled entirely through jQuery event handling and real-time DOM manipulation.

🎨

Color Theme Selection

jQuery .click() event on color swatches. Uses .data(), .addClass(), .removeClass() to switch themes on the 3D product model.

⚖️

Size / Capacity Picker

Radio input change events. Detects selected value with .val(), updates CSS classes, recalculates price dynamically.

✍️

Live Laser Engraving

jQuery .on("input") event tracks keystroke by keystroke. Updates the projected engraving text on the preview mug in real time.

⭐

Premium Pack Unlock

Simulated transaction with setTimeout. .slideUp()/.slideDown() animations reveal premium controls. .prop("disabled") management throughout.

💾

Save Configuration

Form submit handler (.on("submit")). Builds config object and appends a new card to the saved list using .append() with .fadeIn() animation.

🔤

Font Style Switcher

Three engraving font styles (Modern, Serif, Mono). Toggled via class names using jQuery .removeClass() + .addClass().

📸 Screenshots

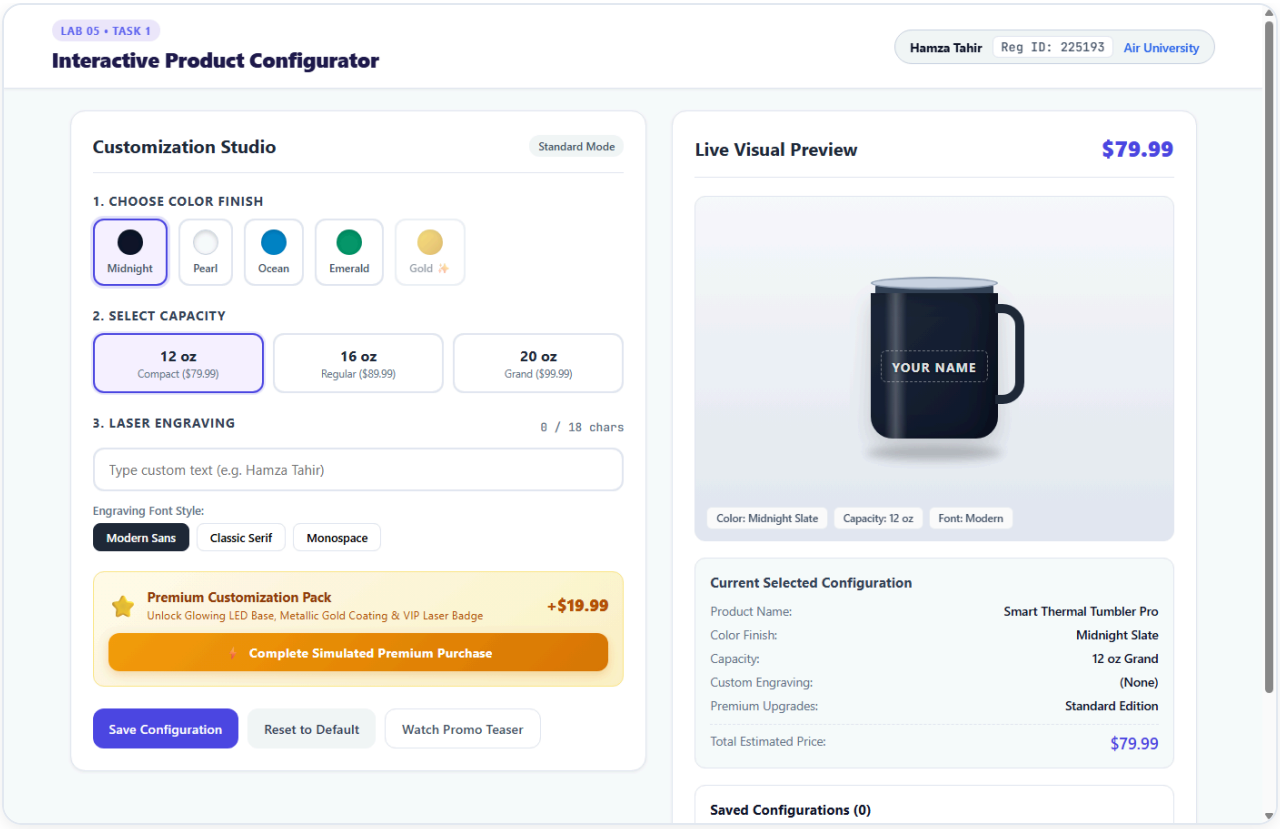


Figure 1.1 — Task 1: Interactive Product Configurator — Initial State showing the Midnight Slate mug with controls

💻 Key jQuery Code — Event Handling & DOM Manipulation

```
// jQuery selectors & event handling — Color Finish Selection
$(".color-swatch").on("click", function() {
    const $swatch = $(this);
    if ($swatch.hasClass("locked")) {
        showAlert("Gold Finish requires Premium Pack!", "warning");
        return;
    }
    $(".color-swatch").removeClass("active");   // Remove from all
    $swatch.addClass("active");                 // Mark selected
    const colorKey = $swatch.data("color");     // jQuery .data()
    $("#product-model").removeClass("theme-midnight theme-pearl theme-ocean theme-emerald")
                       .addClass("theme-" + colorKey); // Dynamic class update
});
```

```
// Real-time Engraving — input event updates live preview
$("#engraving-text").on("input", function() {
    const text    = $(this).val();
    const maxLen  = $(this).attr("maxlength");
    $("#engraving-counter").text(text.length + " / " + maxLen + " chars");
    $("#engraved-label").text(text ? text.toUpperCase() : "YOUR NAME");
});
```

```
// Save Config — Form submit event handler with .append() and .fadeIn()
$("#configurator-form").on("submit", function(e) {
    e.preventDefault();
    const newCard = `<div class="saved-item-card">...</div>`;
    $(newCard).hide().appendTo("#saved-list").fadeIn(400); // Animated append
    $("#saved-count").text(savedConfigurations.length);
});
```

🎯 Concepts Demonstrated (Lab 05 Reference Steps 01–03)

Step 01 — jQuery Introduction & Event Playground

Core jQuery selection with `$(selector)`, .text() updates, input/keyup/click/mouseenter events, .fadeToggle() and .slideToggle() animations.

Step 02 — jQuery Selectors Inspector

ID selector (#id), class selector (.class), element/tag selector (element), .data() attribute reading, .addClass()/.removeClass()/.toggleClass().

Step 03 — DOM Manipulation and Events Studio

.val() for reading form inputs, .text()/.html() for dynamic updates, keydown/input/change event binding, form submit prevention with e.preventDefault().

Files: `Question_1_jQuery_Configurator/index.html` | `styles.css` | `script.js`

TASK 2 / QUESTION 2

Public Product API Integration — jQuery AJAX GET Request

Demonstrates asynchronous AJAX requests, JSON response processing, dynamic rendering, and error handling

📋 Task Overview

Build a two-panel interface that sends a jQuery `$.ajax()` GET request to the public DummyJSON REST API (`https://dummyjson.com/products`). The retrieved product data is dynamically rendered as product cards using `$.each()` and jQuery DOM manipulation. A product can be selected to populate the integrated configurator on the right panel. The interface also features full error handling, live API endpoint display, and a raw JSON inspector.

🔗 How AJAX Works in This Task

When the user clicks "Fetch Public Products", jQuery dispatches an asynchronous HTTP GET request to the DummyJSON API. While waiting, the status bar shows a pulsing "loading" indicator. On success (HTTP 200), `$.each()` iterates through the products array and appends a card per item. On failure, the error callback displays the HTTP error code and disables the render area.

📸 Screenshot

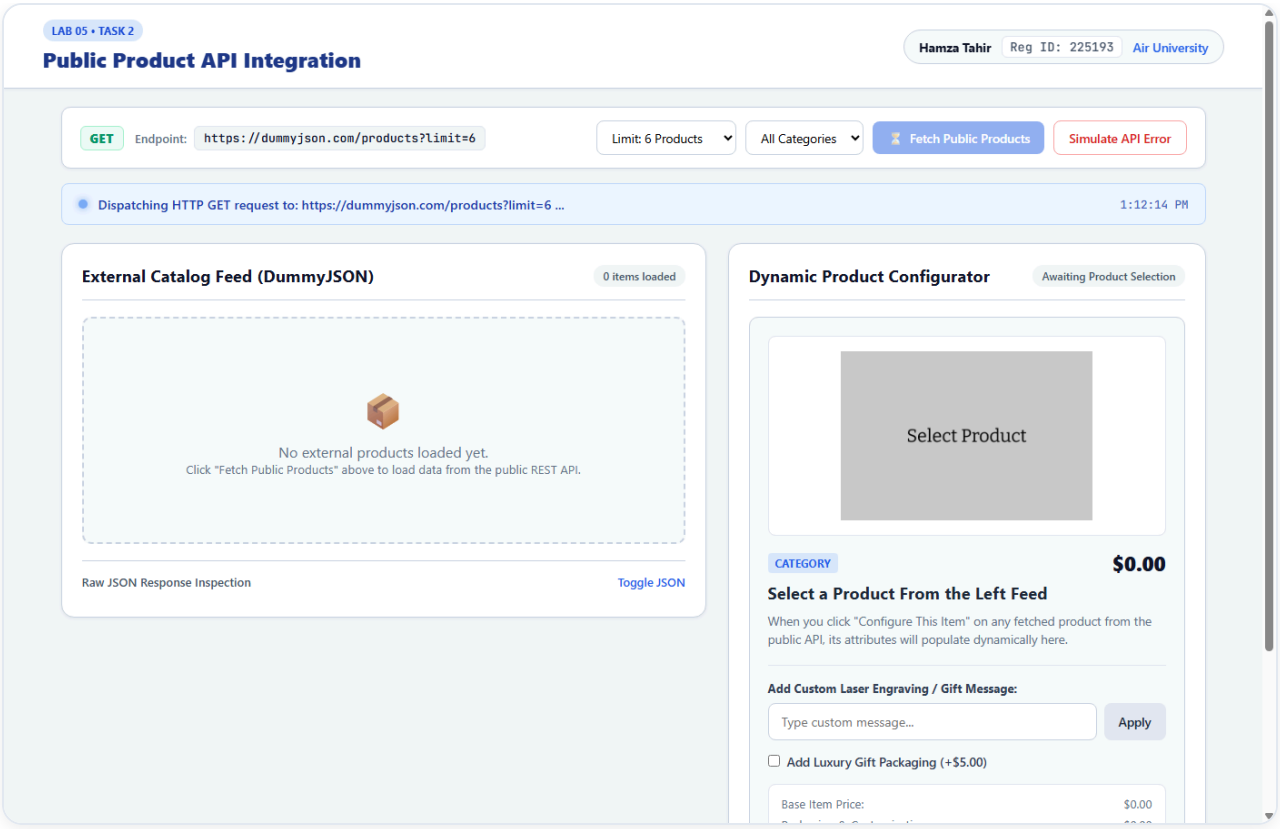


Figure 2.1 — Task 2: Public API Catalog Feed showing AJAX GET request dispatching to DummyJSON API

💻 Core AJAX GET Request Code

```
// jQuery $.ajax() GET request to public DummyJSON API
$.ajax({
    url: `https://dummyjson.com/products?limit=${limit}`,
    method: "GET",
    dataType: "json",
    timeout: 10000,
    success: function(response) {
        console.log("%c[AJAX 200 OK]", "color:#16a34a;", response);
        fetchedProducts = response.products;
        renderCatalog(fetchedProducts);   // Build product cards
        setStatus("success", "Loaded " + response.products.length + " products.");
    },
    error: function(xhr, status, error) {
        console.error("%c[AJAX Error]", "color:#dc2626;", { status: xhr.status, error });
        setStatus("error", "HTTP " + xhr.status + ": " + xhr.statusText);
    }
});
```

```
// Dynamic catalog rendering using jQuery $.each()
function renderCatalog(products) {
    const $container = $("#catalog-products-container").empty();
    $.each(products, function(index, product) {
        $container.append(`
```

```
            <div class="catalog-item-card">
                <img src="${product.thumbnail}" alt="${product.title}">
                <h4>${product.title}</h4>
                <span class="card-price">$${product.price}</span>
                <button class="btn-select-prod" data-id="${product.id}">Configure &rarr;</button>
            </div>
        `);
    });
}
```

🎯 Concepts Demonstrated (Lab 05 Reference Steps 04–05)

> **Step 04 — JSON Basics & Student Directory**
> Parsing JSON response objects, mapping arrays into DOM elements, $.each() iteration, dynamic card generation.

> **Step 05 — AJAX with a Public GET API**
> jQuery $.ajax() method, GET request to DummyJSON, response.products iteration, live search filtering, request state feedback (loading/success/error indicators).

API Endpoint Used

Attribute	Detail
Method	GET
Base URL	`https://dummyjson.com/products`
Query Params	`?limit=N` (4, 6, or 10 products)
Response Format	JSON `{ products: [...], total, limit, skip }`
Error Handling	Simulated via `/non-existent-api-endpoint-404`

Files: `Question_2_Public_API_Configurator/index.html` | `styles.css` | `script.js`

TASK 3 / QUESTION 3

Student Management System — Express.js REST API + jQuery POST

Full-stack: Express.js backend with GET & POST endpoints, CORS middleware, jQuery AJAX frontend

📋 Task Overview

Build a complete full-stack application. The backend is an Express.js REST API server running on `localhost:3005`. It exposes two endpoints: **GET /api/students** returns all student records as JSON, and **POST /api/students** receives a new student object, validates required fields, appends it to the in-memory store, and responds with HTTP 201 Created. CORS middleware (`cors` npm package) is applied so the separate jQuery frontend can communicate cross-origin without browser security errors.

📸 Screenshots

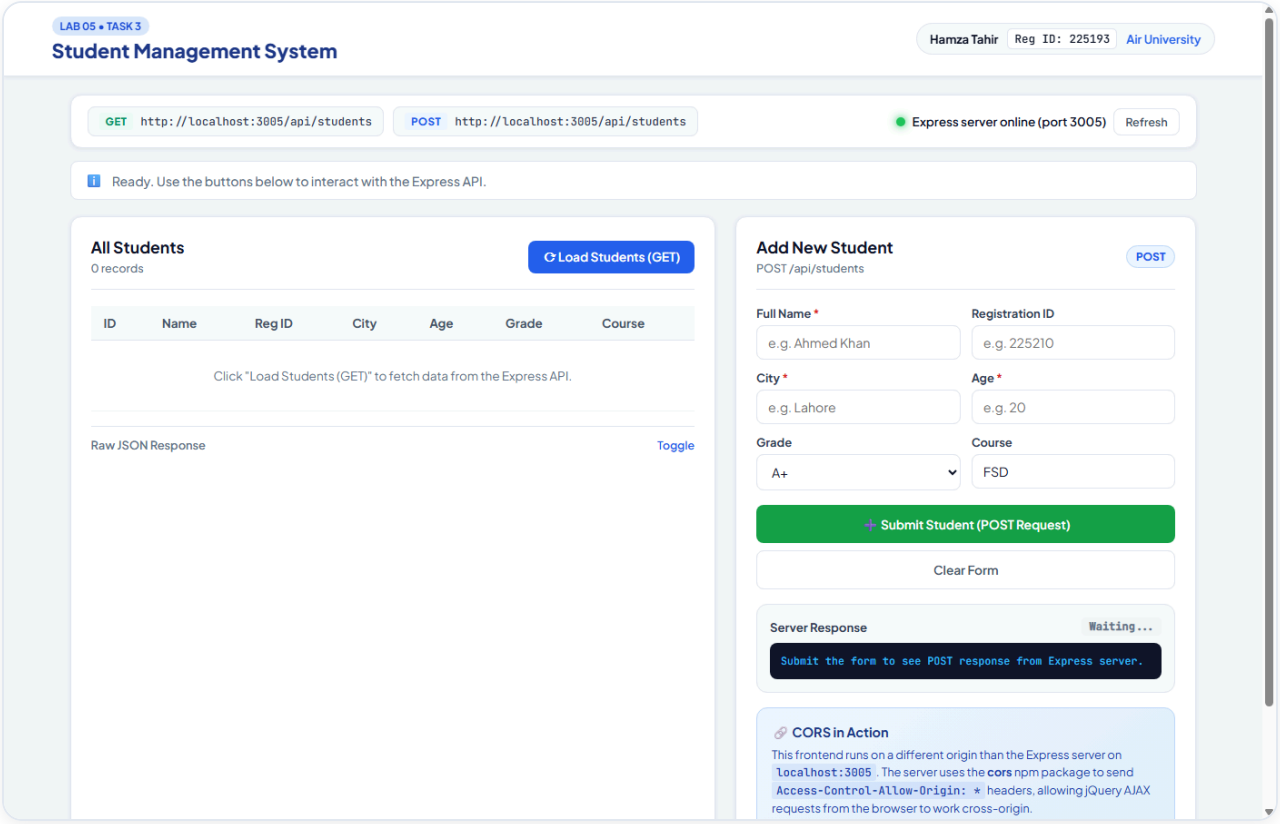


Figure 3.1 — Task 3: Student Management System Frontend with Express server online (green dot), GET table panel & POST form panel

💻 Express Backend Server Code

```
const express = require("express");
const cors    = require("cors");
const app     = express();

// In-memory student data store
let students = [
    { id: 1, name: "Hamza Tahir", regId: "225193", city: "Islamabad", age: 20, grade: "A" },
    { id: 2, name: "Ali Hassan",   regId: "225101", city: "Lahore",     age: 21, grade: "B" }
];

app.use(cors());           // Enable CORS for all origins
app.use(express.json());  // Parse JSON request bodies

// GET /api/students — Return all records
app.get("/api/students", function(req, res) {
    res.status(200).json({ success: true, count: students.length, students: students });
});

// POST /api/students — Add new student
app.post("/api/students", function(req, res) {
    const body = req.body;
    if (!body.name || !body.city || body.age === undefined) {
        return res.status(400).json({ success: false, message: "name, city and age required" });
    }
    const newId = Math.max(...students.map(s => s.id)) + 1;
    const newStudent = { ...body, id: newId, age: Number(body.age) };
    students.push(newStudent);
    res.status(201).json({ success: true, message: "Student added.", student: newStudent });
});

app.listen(3005, () => console.log("Running at http://localhost:3005"));
```

💻 jQuery Frontend AJAX (GET + POST)

```
// AJAX GET request to Express backend
$("#btn-get-students").on("click", function() {
    $.ajax({
        url: "http://localhost:3005/api/students",
        method: "GET", dataType: "json",
        success: function(data) {
            $.each(data.students, function(i, s) {
                $("#students-tbody").append(`<tr><td>${s.id}</td><td>${s.name}</td>...</tr>`);
            });
        }
    });
});

// AJAX POST request — Submit new student form
$("#add-student-form").on("submit", function(e) {
    e.preventDefault();
```

```
    const payload = { name: $("#f-name").val(), city: $("#f-city").val(), age: $("#f-age").val() }
    $.ajax({
        url: "http://localhost:3005/api/students",
        method: "POST",
        contentType: "application/json",
        data: JSON.stringify(payload),
        success: function(data) {
            console.log("201 Created:", data.student);
            // Append new row to table, show JSON response
        }
    });
});
```

✅ API Test Results (Live Verification)

GET /api/students — HTTP 200 OK

Server returned 5 seeded students + 1 dynamically added via POST test. Response: `{"success":true,"count":6,"students":[...]}`

POST /api/students — HTTP 201 Created

Sent: `{"name":"Ahmed Raza","regId":"225200","city":"Peshawar","age":21,"grade":"A"}`
Server responded: `{"success":true,"message":"Student record added successfully.","student":{"id":6,...}}`

API Response JSON (Verified via Node.js)


```
{
  "success": true,
  "count": 6,
  "students": [
    { "id": 1, "name": "Hamza Tahir",  "regId": "225193", "city": "Islamabad", "age": 20, "grade":
    { "id": 2, "name": "Ali Hassan",   "regId": "225101", "city": "Lahore",    "age": 21, "grade":
    { "id": 3, "name": "Sara Ahmed",   "regId": "225122", "city": "Karachi",   "age": 22, "grade":
    { "id": 4, "name": "Usman Malik",  "regId": "225145", "city": "Rawalpindi","age": 20, "grade":
    { "id": 5, "name": "Zainab Raza",  "regId": "225167", "city": "Multan",    "age": 21, "grade":
    { "id": 6, "name": "Ahmed Raza",   "regId": "225200", "city": "Peshawar",  "age": 21, "grade":
  ]
}
```


🎯 Concepts Demonstrated (Lab 05 Reference Steps 07–09)

Step 07 — Express GET API

Setting up Express app, app.get() route handler, res.json() to return data, server startup with app.listen().

Step 08 — Express POST API

app.post() route, express.json() middleware, req.body parsing, server-side validation, res.status(201) response.

Step 09 — CORS Student Application

cors npm package, cross-origin AJAX from frontend, jQuery $.ajax POST with contentType: "application/json", JSON.stringify(payload), real-time DOM update on success.

📁 Project File Structure

```
FSD_Lab_05_225193/
├── Question_1_jQuery_Configurator/
│   ├── index.html      — HTML with form controls & mug canvas
│   ├── styles.css      — Theming, mug 3D model CSS, responsive layout
│   └── script.js       — jQuery selectors, events, DOM manipulation
│
├── Question_2_Public_API_Configurator/
│   ├── index.html      — API control bar, catalog feed, configurator panel
│   ├── styles.css      — Status banner, card grid, JSON inspector
│   └── script.js       — $.ajax GET, $.each rendering, error handling
│
└── Question_3_Express_Backend/
    ├── backend/
    │   ├── server.js   — Express app with GET & POST /api/students
    │   └── package.json— express & cors dependencies
    └── frontend/
        ├── index.html  — Student table + Add Student form
        ├── styles.css  — Panel layout, table, status indicators
        └── script.js   — $.ajax GET/POST with full error handling
```

How to Run Task 3: Open a terminal in `Question_3_Express_Backend/backend/` and run `node server.js`. Then open `frontend/index.html` in a browser. The green health dot confirms the server is online.